

Table 3: Ablation and scaling on threshold of STM with Gemma 2 IT 2B on MBPP task. Percentages indicate the ratio of tokens filtered from training data. STM performs generally the best with 20% to 24% of high ppl tokens filtered.

Configuration	BWT(%)	TI(%)
STM _{$\tau=2.5, high$}	0.4	0.0
STM _{$\tau=2.5, random$}	-8.6	-15.6
STM _{$\tau=2.5, low$}	-7.9	-18.7
Baseline Fine-tuning	-38.2	-25.2
STM _{$\tau=1000$} (6.26%)	-2.9	-11.4
STM _{$\tau=25$} (12.34%)	-2.5	-8.8
STM _{$\tau=10$} (15.1%)	-0.7	-10.4
STM _{$\tau=2.5$} (23.8%)	0.4	0.0
STM _{$\tau=1.5$} (26.1%)	-0.3	-0.5
STM _{9B$\tau=2.5$} (23.8%)	-3.8	-7.3

Table 4: STM applied on full weight fine-tuning (FWFT), DoRA and LoRA with MBPP on Gemma 2 IT 2B. STM can enhance all three kinds of fine-tuning techniques with much better preservation of non-target task capability and target task improvement.

Configuration	BWT (%)	TI(%)
FWFT	-31.87	-27.98
FWFT + STM _{$\tau=2.5$}	-0.13	-8.81
LoRA	-38.19	-21.76
LoRA + STM _{$\tau=2.5$}	0.42	0.0
DoRA	-8.54	-15.2
DoRA + STM _{$\tau=2.5$}	-0.01	0.04

5.2 Is larger-sized LLM a better token filter?

In addition to that, STM uses initial model perplexity for token selection, we investigate an alternative token selection model for STM.

Cross-scale filtering. We explore perplexity assessment using larger models within the same model family (*e.g.*, Gemma 2 IT 9B) to guide token selection for smaller models (*e.g.*, Gemma 2 IT 2B), denoted as STM_{9B $\tau=2.5$} . This approach investigates whether token selection benefits from a more robust understanding of larger models, potentially offering a form of knowledge distillation through perplexity-based filtering.

Table 3 shows that both STM and STM_{9B} perform better than Baseline Fine-tuning. However, scaling the perplexity model does not surpass STM with masking created by its own (2B), likely because larger models assign lower perplexity to tokens smaller models find challenging. Therefore, the optimal strategy should always be to use the same model for token selection. We also investigated other alternatives such as a learnable token selection, please see Appendix D.4.

5.3 Optimal threshold selection for STM

A key hyperparameter of STM is the perplexity threshold to filter tokens from the supervised fine-tuned (SFT) model. We explore the existence of an optimal threshold that maximizes performance. The bottom part of Table 3 reveals that filtering approximately 24% of tokens yields the best results. This optimal threshold demonstrates consistent benefits across both target and non-target datasets, including GSM8K, ARC, MATH, and BIRD, showing its robustness and generalization similar to Self-Output data. For further validation of the generalization of such settings, please refer to Table 11 in Appendix D that such settings apply to different model families as well.

5.4 Is forgetting affected by smaller learning rates?

Following training settings in [9, 37], learning rate (lr) would be one reason that harms the performance of Baseline Fine-tuning severely (but not for STM and LLM generated data baselines). Therefore, we conducted additional learning rate sweeps down to $1e-7$ for Llama 3 8B Instruct and Gemma 2 IT 2B on the MBPP dataset. As Table 5 shows, we found that Baseline Fine-tuning improves at smaller learning rates (*e.g.*, TI: $-10.6\% \rightarrow 1.33\%$), but the forgetting problem (BWT) persists (*e.g.*, -1.6%). In contrast, STM consistently outperforms Baseline across all learning rates with stronger target task performance and substantially lower forgetting. Notably, STM achieves better BWT and stable performance even at small learning rates (*e.g.*, TI: 2.23%, BWT: +1.39% at $1e-7$). These results demonstrate that STM is more robust to learning rate choices, requires less learning rate tuning, and achieves more stable training dynamics, highlighting its practical advantage in real-world fine-tuning scenarios.

Table 5: Learning rate (lr) sweeping results of training Llama 3 8B Instruct (Llama 3 8B-IT) and Gemma 2 IT 2B on the MBPP dataset. Note that only BWTs of Baseline Fine-tuning (Baseline) with the best TI are calculated. The results show the higher robustness of STM to learning rate choices while still forgetting less or nothing.

Llama 3 8B-IT	lr	BWT(%)	TI(%)
BASILINE	1E-4	-	-10.6
BASILINE	2E-5	-34.7	-2.23
BASILINE	5E-6	-	0.9
BASILINE	1E-6	-	-4.47
BASILINE	5E-7	-	-1.3
BASILINE	1E-7	-1.6	1.33
STM	1E-4	1.8	-3.58
STM	2E-5	0.2	3.2
STM	5E-6	-0.1	2.68
STM	1E-6	0.53	3.12
STM	5E-7	1.23	3.12
STM	1E-7	1.39	2.23

Gemma 2 IT 2B	lr	BWT(%)	TI(%)
BASILINE	2E-5	-38.2	-15.5
BASILINE	5E-6	-	-4.0
BASILINE	1E-6	-	-17.6
BASILINE	1E-7	-4.7	-0.53
STM	2E-5	-0.3	-0.5
STM	5E-6	-1.1	-3.0
STM	1E-6	-0.35	-1.5
STM	1E-7	0.51	0.7

Table 6: Self-bleu score of Baseline Fine-tuning and STM training. The results show that both training lead to lower diversity, but masking would not reduce further diversity but less forgetting.

Llama 3 8B Instruct	lr	self-bleu	accuracy(%)	BWT (%)
Original	-	20.47±13	44.75	-
Baseline Fine-tuning	1E-7	40.29±19	56.5	-1.6
STM _{r=2.5}	1E-7	40.77±17	58.25	1.39

5.5 STM generalization on different fine-tuning strategies

As STM is a masking technique applied to target data during the fine-tuning stage, it should be orthogonal to the fine-tuning techniques and improve the performance degradation. Thus, we experiment with how STM improves practical fine-tuning strategies like Full-Weight Fine-tuning (FWFT), Low-rank adaptation (LoRA), and Weight-decomposed LoRA (DoRA) [22]. As Table 4 shows, all of the fine-tuning techniques are further improved by STM, which indicates STM’s generalization on different fine-tuning strategies.

5.6 Does masking in STM lead to lower diversity of token generation?

Masking in STM reflects a possibility that model learns to generate a token distribution of higher kurtosis (lower diversity) by masking more tokens not for training. Thus, we experiment with a Llama 3 8B Instruct model trained on MBPP coding dataset and tested on a sample of 100 MATH questions. Each instance’s reasoning response is generated 4 times, and average self-bleu scores are calculated. As Table 6 shows, Baseline Fine-tuning and STM share similar diversity, while Baseline Fine-tuning leads to more forgetting, which STM avoids. Therefore, it is more likely that training typically harms diversity, but STM could enhance Baseline Fine-tuning with less forgetting.

6 Analysis and discussion

In our experiments, we found that STM’s effectiveness is applicable across different datasets, models, and fine-tuning strategies. In this section, we conduct several analyses on the model training process and model changes with STM to investigate the reason that reduces performance degradation.

Low training loss from low perplexity training. Training low perplexity data leads to a low training loss intuitively, and the model can converge within fewer training epochs. Figure 4 showcases the training procedure on MBPP training data of different perplexities. The results show that training with STM or synthesized data yields much lower training loss, resulting in fewer updates of model weight for changing capabilities of non-target tasks.

Subtle LoRA weight changed from low perplexity training. We investigate the relationship between token-level perplexity and characteristics of weight updates in LoRA. Our analysis focuses

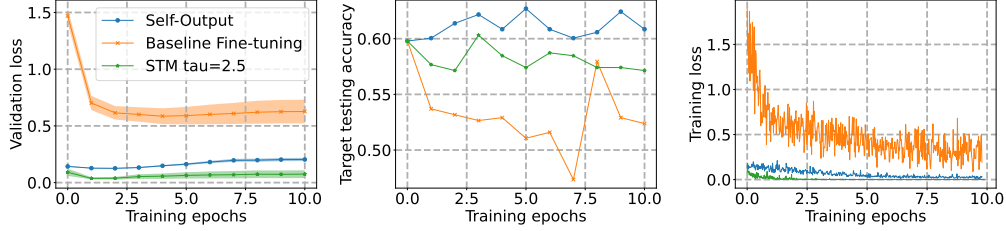


Figure 4: MBPP target task testing accuracy, validation and training loss of baseline finetuning with ground truth data, finetuning with Self-Output data, and STM strategy of perplexity filtering threshold=2.5 for Llama 3 8B Instruct. STM and self-output training yield better performances with much lower training and validation loss because of low-perplexity training.

Table 7: L2 norm of ΔW signifies a larger update from the original weight W . STM shows a small model ΔW , indicating STM training can occur with few changes on the model’s original capabilities.

Models tuned on MBPP	Self-Output	Rephrase	Ground Truth	STM + Ground Truth
Llama 3 8B Instruct	6.53	7.31	17.75	0.55
Gemma 2 IT 2B	4.03	5.78	5.69	0.45

on understanding how different types of training data affect the magnitude and dimensionality of parameter adjustments in transformer-based language models. Given that LoRA effectively learns offset weights ΔW to the original model parameters W , we calculate the L2 norm of ΔW to quantify the degree of deviation introduced by each fine-tuning dataset. As shown in Table 7, fine-tuning with Self-Output data consistently results in smaller L2 norm of weight updates compared to Rephrase and Ground Truth data, while STM produces the smallest updates (0.45-0.56) across all models. Such minimal perturbations suggest that both approaches, particularly STM, help preserve the model’s original capabilities. Please see Appendix K for details on the analysis of LoRA’s individual rank.

High perplexity training leads to more degradation with similar weights updated. Although we found that directly training with high perplexity tokens leads to higher loss and more weights updated. Applying regularization techniques can reduce model weight changes to mitigate non-target task degradation. Hence, we apply alternative regularization techniques to fine-tune with high-perplexity ground truth data to mitigate non-target task performance degradation. Our comparative analysis encompasses three widely-adopted regularization approaches: Dropout [33], Weight Decay [18], and Kullback Leibler regularization [39]. These methods have demonstrated effectiveness in preventing models from overfitting. Table 8 shows that under similar model weight changes, models applied with STM still outperform the high perplexity data trained models with regularization, indicating the performance degradation results from high perplexity training instead of purely model overfitting. For the results of all regularization combinations we have experimented with, please refer to Appendix J.

7 Related work

Instruction-tuning. Instruction following [5, 28] is widely used to align responses of LLMs with target task values. Practically, it requires instruction tasks dataset to fine-tune LLMs, where each dataset consists of instructions and desired responses. Hence, the performance of instruction-tuning heavily relies on the quality of instruction data such as context richness [41, 45, 50]. In addition, Ghosh et al. [9] investigated the limitation of instruction-tuning of LLMs about the catastrophic forgetting from pattern-copying behaviors and hallucinations with Lora/full fine-tuning. These works give us a good start on how performance degrades in terms of models’ response behaviors and benchmarks when fine-tuning with instruction following dataset.

Using LLM-generated data for instruction fine-tuning. As Chung et al. [5], Wang et al. [41] showed, LLMs break down easily after training with different tasks. Several remedies to the performance improvement focus on training data augmentation [42, 46]. For instance, [31] uses much larger LLMs (*e.g.*, GPT-4 and Claude) to generate responses of questions as training labels, which improves the performance on both the target task and other non-target tasks. However, such a distillation method neglects the correctness of generated labels, so that more incorrect responses could be trained as the amount of generated data increases, and thus, using equal or smaller-sized